

Sprint 1 AI Usage Log

SOEN 341 Software Process, Fall 2026 | CareerConnect - Group A

Team member	Vladislav Shkuratov (GitHub: vshkuratov4)
Repository	github.com/doryan-gonin/CareerConnect-GroupA
Date	September 27, 2026
AI tools used	1) Claude (claude.ai chat, Project workspace) for explanations, planning, code suggestions and debugging. 2) Claude Code (VS Code extension) for generating the signup/login frontend and diagnosing a startup error.
My tasks this sprint	Issue #45 (US-01.04: Fix issues with auth), issue #36 (US-01.01: UI signup and login pages)

All AI output listed below was read, run or reviewed by me before it was committed, and I am responsible for it. Where the AI made a mistake, the entry says so. **Status note:** endpoint-level tests at /docs for tasks 2.2, 2.3 and 2.4 (lockout, refresh/logout, password rules) were still to be completed when this log was written, so those entries only list the validation done so far.

Summary

ID	Task	Tool	Decision
1.1	Understanding the project, Agile/Scrum and GitHub workflow	Claude	Accepted
1.2	Backend codebase walkthrough (FastAPI, JWT, SQLAlchemy)	Claude	Accepted
2.1	JWT secret key moved to an environment variable (#45)	Claude	Modified
2.2	Failed login attempts and temporary lockout (#45)	Claude	Modified
2.3	Token verification, logout and token refresh (#45)	Claude	Accepted
2.4	Password strength rules (#45)	Claude, Claude Code	Modified
2.5	Debugging startup errors (uv, import typo, .env typo)	Claude, Claude Code	Accepted
3.1	Git branching, pull request and PR text help	Claude	Modified
4.1	Signup and login UI (#36)	Claude, Claude Code	Modified (part rejected)

Log Entries

Task 1.1: Understanding the project, Agile/Scrum and the team's GitHub workflow

Purpose of AI Use	Learning and research. I had not attended any lectures before Sprint 1, so I needed to understand the course material, the repository, the issue board and what work was left.
Chat Link or Prompt/Response	Claude chat, see Appendix A, prompts 1 to 3 (I shared the week 1 and 2 slides, the Sprint 1 handout, and screenshots of the repo, branches, issues and project board).
AI-Suggested Content	Plain-language explanations of Git and GitHub terms (branches, pull requests, merge conflicts, issues, milestones, sub-issues), Scrum vocabulary (sprint, backlog, user story, Definition of Ready and Done), a sorted list of the 16 closed and 22 open issues into done, in progress and not started, and suggestions for which tasks I could take.
Validation	I compared the explanations with the course slides, the Sprint 1 handout and the GitHub Issues and Projects pages. One claim was wrong: the AI first said no Team Process Definition existed, but the closed issue list showed #7 (3 of 3 sub-issues done). It corrected this after I shared the closed issues.
Decision	Accepted. Used for understanding only, nothing was copied into a deliverable.
Reflection	Good for getting oriented quickly, but it can misread screenshots and make assumptions, so I checked its claims against the repository before acting on them.
Responsible Person	Vladislav Shkuratov

Task 1.2: Backend codebase walkthrough

Purpose of AI Use	Understanding code I had not written and had never worked with (FastAPI, SQLAlchemy, JWT, bcrypt).
Chat Link or Prompt/Response	Claude chat, Appendix A, prompts 4 and 5. The AI read the repository files I copied to Google Drive (main.py, database.py, models.py, routers/auth.py, routers/profiles.py, pyproject.toml, and the frontend scaffold).
AI-Suggested Content	A file-by-file explanation of the backend, an explanation of JWT tokens, password hashing and the database layer, how to run the server and use the /docs test page, and a mapping of the gaps in auth.py to the five tasks in issue #45. It also noted that the profile endpoints take the user id from the URL without checking a token.
Validation	I compared the explanation with the actual code in my editor and checked the request fields (email_address, password) in models.py. I ran the backend locally and it started.
Decision	Accepted.
Reflection	This made the codebase readable in one sitting. I still need to be able to explain the auth flow myself in the lab, so I re-read auth.py afterwards.
Responsible Person	Vladislav Shkuratov

Task 2.1: JWT secret key moved to an environment variable (#45)

Purpose of AI Use	Code generation for the task 'Set up JWT secret key as an environment variable'.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompt 6.
AI-Suggested Content	Use python-dotenv, a .env file that is ignored by Git, a committed .env.example, and reading the key with os.getenv in auth.py, with a RuntimeError if it is missing so the server refuses to start without a secret.
Validation	I reviewed the code, ran git status to confirm .env was not tracked, and ran the server. The first run failed with the RuntimeError, which showed that my .env had a wrong variable name (JWT_SECRET_KET instead of JWT_SECRET_KEY). I fixed it and the server started.
Decision	Modified. I fixed typos in the error message text and in the .env variable name.
Reflection	The startup guard was useful because it turned a silent misconfiguration into an obvious error. The AI code was correct, the mistakes were my typos.
Responsible Person	Vladislav Shkuratov

Task 2.2: Failed login attempts and temporary lockout (#45)

Purpose of AI Use	Code generation for the task 'Keep track of failed attempts and block user for a set amount of time'.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompt 7.
AI-Suggested Content	Two new columns on the User table (failed_attempts, locked_until), constants for 5 attempts and 15 minutes, and changes to login_user: return 403 while locked, count wrong passwords, lock at the limit, reset the counters on success.
Validation	I reviewed the code and committed it. Later the AI pointed out a bug in its own suggestion: SQLite returns datetimes without timezone information, so comparing locked_until with a timezone-aware datetime.now would raise a TypeError on the sixth attempt. I applied the corrected comparison and pushed it. I also deleted the old local database because SQLite does not add columns to an existing table. The end-to-end test (five wrong passwords, then a 403) is still to be run.
Decision	Modified after the AI's own bug was found.
Reflection	The AI wrote this without running it, which is how the datetime bug slipped in. Reviewing AI code for edge cases like this matters more than the first draft.
Responsible Person	Vladislav Shkuratov

Task 2.3: Token verification, logout and token refresh (#45)

Purpose of AI Use	Code generation for the tasks 'Add a logout endpoint that invalidates the token' and 'Refresh token while user is active'.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompt 8.
AI-Suggested Content	A RevokedToken table, a unique id (jti) inside every token, a get_token_payload / get_current_user dependency using HTTPBearer, POST /api/auth/logout that stores the token id as revoked, and POST /api/auth/refresh that revokes the old token and returns a new one.
Validation	I reviewed the code, added it to auth.py and models.py, and the server starts with the new code without errors. The endpoint tests in the PR description (refresh, then reuse of the old token returns 401, logout, then reuse returns 401) are still to be run.
Decision	Accepted.
Reflection	Logout on a stateless JWT needed a design idea (a revocation list), which the AI explained well. I still have to verify it works by testing it.
Responsible Person	Vladislav Shkuratov

Task 2.4: Password strength rules (#45)

Purpose of AI Use	Code generation for the task 'Enforce password security criteria'.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompt 9. Claude Code reviewed the resulting diff.
AI-Suggested Content	A validate_password_strength function (8+ characters, uppercase, lowercase, number, special character, and a 72-byte maximum because of bcrypt) called in register_user before hashing.
Validation	When I reviewed the uncommitted diff, two defects were visible: the import re line was missing, and the @router.post('/register') decorator had ended up above the helper function instead of above register_user (my placement instruction was ambiguous), which would have broken registration. Claude Code had already corrected both, I read the diff to confirm, and moved the fix onto the auth branch. The rules themselves still need the /docs test (weak password gives 400, strong gives 201).
Decision	Modified.
Reflection	Ambiguous instructions produce subtly broken code. Reading the git diff before committing caught it.
Responsible Person	Vladislav Shkuratov

Task 2.5: Debugging startup errors

Purpose of AI Use	Debugging. Three separate errors stopped me from running the backend.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompts 10 to 12 (screenshots and terminal output). Claude Code diagnosed the .env variable name.
AI-Suggested Content	(a) uv was not installed, so it gave the install command and told me to restart the terminal. (b) ImportError caused by writing load_detenv instead of load_dotenv. (c) The RuntimeError from the misspelled .env variable name.
Validation	After each fix I re-ran the server, and the last run printed that the server had started at 127.0.0.1:8000.
Decision	Accepted.
Reflection	The traceback was long but the real message was on the last line, which is a habit I will keep.
Responsible Person	Vladislav Shkuratov

Task 3.1: Git branching, pull request and PR text help

Purpose of AI Use	Process guidance for a workflow I had not used before.
Chat Link or Prompt/Response	Claude chat, Appendix A, prompts 13 to 16.
AI-Suggested Content	Commands for pulling main, creating a branch per task, staging only the intended files, pushing, using git stash to move a fix to the right branch, exiting the nano merge editor, opening pull requests, and drafts of the PR title and description.
Validation	I checked the results with git status, git log and GitHub's 'Able to merge' check, and kept files like .env and the database out of the commits. The first PR text did not follow the team's PR template, so I asked for a version in the exact template format.
Decision	Modified. PR text was reformatted to match the template.
Reflection	Useful, but the team's template and rules take priority over the AI's suggested format.
Responsible Person	Vladislav Shkuratov

Task 4.1: Signup and login UI (#36)

Purpose of AI Use	Implementation of the frontend for issue #36 (signup and login pages, React with Tailwind).
Chat Link or Prompt/Response	I wrote the prompt with Claude chat (Appendix B contains the full prompt) and ran it in Claude Code.
AI-Suggested Content	SignupForm, LoginForm, PasswordRequirements and LoggedInScreen components, src/api/auth.js, src/utis/passwordRules.js, a Vite proxy for /api, and a rewritten App.jsx and index.css. A placeholder 'logged in' screen with a Logout button. Claude Code also edited backend/routers/auth.py even though the prompt said not to.
Validation	I ran the backend and frontend together and completed a login in the browser, ending on the logged-in placeholder screen. I listed the changed files with git status and committed only src/frontend. I reviewed the auth.py diff, kept it out of this branch and moved it to the auth branch, where it was a real fix (see Task 2.4).
Decision	Modified. Frontend accepted. The unrequested backend edit was rejected on this branch and relocated.
Reflection	The code works, but I need to read LoginForm.jsx and auth.js closely so I can explain them in the lab. Restricting what the AI may edit did not fully hold, so I always review the diff.
Responsible Person	Vladislav Shkuratov

Appendix A - Claude chat (claude.ai)

Note: I am not providing a public link to the conversation because it contains personal documents and private team messages. The list below is a condensed paraphrase of my prompts, not a verbatim copy. The full conversation can be shown to the TA during the lab on request.

Condensed list of my prompts, in order:

1. Explained I had never attended class, shared the week 1 and 2 slides, the Sprint 1 handout, screenshots of the repository and branches, the issue lists and the team chat, and asked for a from-zero explanation of GitHub, the tasks, the code and what was done and not done.
2. Said I understood Git and Scrum but not what was going on in the repository's issues, and asked what is done, what is not, and how to find and claim issues.
3. Asked how to clone the repository in VS Code, then how to share the code with the AI (I chose copying the project files to Google Drive).
4. Shared the Drive folder and the project board and asked whether to build the signup UI with plain HTML, TypeScript or a framework.
5. Shared my issue #45 and asked for a file-by-file explanation of the backend and how to start on the auth fixes.
6. Asked how to pull the latest code, create a branch, and start task 5 (JWT secret in an environment variable). Followed up with a screenshot of my git status and .env.example.
7. Asked for the next task after the secret key: failed attempts and lockout, then how to commit it.
8. Asked for the logout and refresh tasks.
9. Asked what was still missing from issue #45 and for the password rules.
10. Shared the uv not found error. 11. Shared the load_detenv ImportError traceback. 12. Shared the RuntimeError traceback and a screenshot of the running server.
13. Asked how to run and test the features without a frontend. 14. Asked how to create a second branch and pause the auth work. 15. Shared nano and pager screens and git diff output. 16. Asked for the PR titles and descriptions in the team template.
17. Asked for a short prompt for Claude Code to build the frontend (Appendix B).

Appendix B - Prompt given to Claude Code (issue #36)

I'm a beginner with React and I'm working on a team project (CareerConnect, a job-search tracking app). My task is GitHub issue #36 "US-01.01: UI signup and login pages". Please build it and explain what you're doing in plain language as you go, since I want to learn.

STACK (already set up in src/frontend, don't change it):

- React 19 + Vite + Tailwind CSS 4, plain JavaScript (.jsx), NOT TypeScript
- The current App.jsx / App.css are the untouched Vite demo template. Replace them. Keep `@import "tailwindcss";`` in index.css and remove the demo styles.
- No router or other libraries are installed. Ask me before adding any dependency.
- Use Tailwind classes for styling, no separate CSS files.

BACKEND (FastAPI, runs at `http://127.0.0.1:8000`, do NOT modify anything in src/backend):

- POST `/api/auth/register`, JSON body `{"email_address": "...", "password": "..."}`
 - 201 returns `{id, email_address, created_at}`
 - 400 returns `{"detail": "..."} (email already exists, or weak password)`
- POST `/api/auth/login`, same JSON body
 - 200 returns `{"access_token": "...", "token_type": "bearer"}`
 - 401 "Invalid email or password."
 - 403 means the account is temporarily locked after 5 failed attempts
 - 422 for an invalid email format (its "detail" is a list, not a string, so handle both shapes)
- Password rules enforced by the backend: at least 8 characters, an uppercase letter, a lowercase letter, a number, and a special character. Validate these in the UI too and show the requirements.
- The backend has no CORS setup, so configure a Vite dev proxy that forwards `/api` to `http://127.0.0.1:8000` and call relative URLs like `fetch("/api/auth/login")` instead of touching the backend.

WHAT TO BUILD:

1. A Signup page: email, password, confirm password, submit button, loading state, and clear error messages from the server. On success, send the user to the login page with a "Account created" message.
2. A Login page: email, password, submit button, loading state, error messages (including the lockout message). On success, store the `access_token` in `localStorage` and show a simple placeholder "logged in" screen with a Logout button that clears the token.
3. A simple way to switch between the two pages without a router library (a piece of state in `App.jsx` is fine).
4. Split it into small components (for example `SignupForm.jsx`, `LoginForm.jsx`) in `src/frontend/src/components`, and put the fetch calls in one small file (for example `src/frontend/src/api/auth.js`).
5. Clean, modern, responsive design.

WORKFLOW:

- Check that node and npm are installed, then run `npm install` in `src/frontend` and start it with `npm run dev`.
- I'll run the backend in a separate terminal: from the repo root, `uv run fastapi dev src/backend/main.py``.
- Test against the real backend and tell me how to do the same by hand.
- I'm on branch `feature/36-signup-login-ui`. Don't push or commit without asking me, and don't touch `README.md`, `.env`, or any database file.
- At the end, list the files you changed and summarize them so I can write the PR description.

Claude Code's response (files created and the summary of manual test steps) is visible in the Claude Code panel history of the session and in the commit 'Add signup and login UI (#36)' on branch `feature/36-signup-login-ui`.